use a prompt contract like "answer only from context" and require citations per claim. I also add an abstain path when confidence is low or sources conflict.

1.1 Core Pipeline

1.1.1 Ingestion and chunking

Chunking should preserve meaning boundaries. Use headings, paragraphs, and lists as primary split points, then enforce a token budget. Overlap helps recall for facts that span boundaries, but too much overlap increases index size and duplicates in Top-k.

1.1.2 Dense retrieval

Dense retrieval is typically ANN search over embeddings using FAISS, ScaNN, or a managed vector database. Start with cosine similarity or inner product, then measure recall@k on a labeled set. If recall@k is low, the generator cannot fix it.

1.1.3 Prompt assembly and grounding

A grounded prompt should include: user question, retrieved chunks with source IDs, and explicit constraints. I usually add: "If the answer is not in the context, say you don't know." I also ask for short citations like `[doc3#p2]` per sentence.

1.1.4 Observability and evaluation

Log the query, Top-k chunk IDs, similarity scores, and the final answer with citations. Evaluate retrieval (recall@k, MRR) and generation (faithfulness, answer correctness) separately. This separation makes debugging fast.

1.2 Implementation Notes

1.2.1 Minimal reference implementation (pseudocode)

```
# Offline: build index
for doc in corpus:
    chunks = chunk(doc.text, max_tokens=400, overlap=60)
    for c in chunks:
        vec = embed(c.text)
        index.add(vec, metadata={source_id, title, ts, chunk_id})

# Online: answer
q_vec = embed(user_query)
topk = index.search(q_vec, k=6) # returns (chunk_text, metadata, score)
prompt = build_prompt(user_query, topk, require_citations=True, allow_abstain=True)
answer = llm.generate(prompt)
return answer
```